

# Simulation données service public : ADEME

Projet pédagogique SQL/PostgreSQL, construit à partir de données 100% FICTIVES inventées pour l'exercice. Il simule le fonctionnement d'un organisme public de type ADEME (instruction et suivi financier de dossiers d'aides), sans utiliser aucune donnée réelle. Il illustre la hiérarchie récursive, les fonctions de fenêtrage, les sous-requêtes, JSONB/ARRAY, les index, les transactions et les vues.

■■ **Données fictives** : Tous les organismes, agents, montants, dates, SIRET et emails de ce projet sont fictifs et ne correspondent à aucune entité réelle. 'ADEME' sert uniquement de contexte inspirant un schéma réaliste, pas de source de données.

---

Moteur : PostgreSQL

Nombre d'exercices : 15

Lien DB Fiddle : <https://www.db-fiddle.com/f/n7Vqz25KhTNvxMEN8pXuiG/0>

**Mor Talla DIENG**

Data Analyst RH/Finance

# 1. Schéma de la base de données

## categories\_aides

| Colonne             | Définition                                                              |
|---------------------|-------------------------------------------------------------------------|
| id                  | SERIAL PRIMARY KEY                                                      |
| nom                 | VARCHAR(100) NOT NULL                                                   |
| categorie_parent_id | INT REFERENCES categories_aides(id) - auto-référence pour la hiérarchie |

## organismes

| Colonne         | Définition                      |
|-----------------|---------------------------------|
| id              | SERIAL PRIMARY KEY              |
| nom             | VARCHAR(150) NOT NULL           |
| siret           | VARCHAR(14) UNIQUE NOT NULL     |
| secteur         | VARCHAR(100)                    |
| region          | VARCHAR(100)                    |
| taille_effectif | INT CHECK (taille_effectif > 0) |

## agents\_instructeurs

| Colonne | Définition                   |
|---------|------------------------------|
| id      | SERIAL PRIMARY KEY           |
| nom     | VARCHAR(100) NOT NULL        |
| email   | VARCHAR(100) UNIQUE NOT NULL |

## dossiers\_aide

| Colonne         | Définition                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------|
| id              | SERIAL PRIMARY KEY                                                                            |
| organisme_id    | INT REFERENCES organismes(id)                                                                 |
| categorie_id    | INT REFERENCES categories_aides(id)                                                           |
| agent_id        | INT REFERENCES agents_instructeurs(id)                                                        |
| montant_demande | NUMERIC(12,2) CHECK (montant_demande > 0)                                                     |
| montant_accorde | NUMERIC(12,2) CHECK (montant_accorde >= 0)                                                    |
| statut          | VARCHAR(20) DEFAULT 'déposé' CHECK (statut IN ('déposé','en instruction','accepté','refusé')) |
| date_depot      | DATE DEFAULT CURRENT_DATE                                                                     |
| date_decision   | DATE                                                                                          |

|         |        |
|---------|--------|
| details | JSONB  |
| tags    | TEXT[] |

## enveloppes\_budgetaires

| Colonne       | Définition                                                                    |
|---------------|-------------------------------------------------------------------------------|
| id            | SERIAL PRIMARY KEY                                                            |
| categorie_id  | INT REFERENCES categories_aides(id)                                           |
| annee         | INT NOT NULL                                                                  |
| budget_alloue | NUMERIC(14,2) CHECK (budget_alloue > 0)                                       |
| contrainte    | UNIQUE (categorie_id, annee) - une seule enveloppe par catégorie et par année |

## versements

| Colonne        | Définition                        |
|----------------|-----------------------------------|
| id             | SERIAL PRIMARY KEY                |
| dossier_id     | INT REFERENCES dossiers_aide(id)  |
| montant        | NUMERIC(12,2) CHECK (montant > 0) |
| date_versement | DATE DEFAULT CURRENT_DATE         |

## 2. Exercices — commandes, explications, résultats

### Exercice 1 — CTE réursive - hiérarchie complète des catégories d'aides

Notion : WITH RECURSIVE

■ **Objectif** : Afficher toutes les catégories d'aides avec leur niveau de profondeur, en descendant automatiquement dans la hiérarchie parent/enfant, quel que soit le nombre de niveaux.

```
WITH RECURSIVE arbre_categories AS (  
    SELECT id, nom, categorie_parent_id, 1 AS niveau  
    FROM categories_aides  
    WHERE categorie_parent_id IS NULL  
    UNION ALL  
    SELECT c.id, c.nom, c.categorie_parent_id, ac.niveau + 1  
    FROM categories_aides c  
    JOIN arbre_categories ac ON c.categorie_parent_id = ac.id  
    WHERE ac.niveau < 10  
)  
SELECT * FROM arbre_categories ORDER BY niveau, nom;
```

**Explication** : L'ancrage part des catégories sans parent (niveau 1). La partie réursive rejoint la CTE à elle-même pour retrouver les enfants, niveau après niveau, jusqu'à épuisement.

| nom                    | parent_id | niveau |
|------------------------|-----------|--------|
| Bâtiment               | None      | 1      |
| Énergie                | None      | 1      |
| Efficacité énergétique | 1         | 2      |
| Énergies renouvelables | 1         | 2      |
| Isolation              | 2         | 2      |
| Éolien                 | 3         | 3      |
| Solaire photovoltaïque | 3         | 3      |

### Exercice 2 — Fonctions de fenêtrage - classement des montants par région

Notion : RANK() OVER (PARTITION BY ... ORDER BY ...). AVG() OVER

■ **Objectif** : Classer chaque dossier par montant demandé au sein de sa région, tout en gardant chaque ligne individuelle visible (contrairement à un simple GROUP BY).

```
SELECT o.region, o.nom AS organisme, d.montant_demande,  
       RANK() OVER (PARTITION BY o.region ORDER BY d.montant_demande DESC) AS rang_region,  
       ROUND(AVG(d.montant_demande) OVER (PARTITION BY o.region), 2) AS moyenne_region  
FROM dossiers_aide d  
JOIN organismes o ON d.organisme_id = o.id  
ORDER BY o.region, rang_region;
```

**Explication** : PARTITION BY région recommence le classement à 1 pour chaque région, sans fusionner les lignes contrairement à GROUP BY.

| region   | organisme        | montant | rang | moyenne_region |
|----------|------------------|---------|------|----------------|
| Bretagne | Ville de Rennes  | 78000.0 | 1    | 51000.0        |
| Bretagne | EcoBât Solutions | 45000.0 | 2    | 51000.0        |

|                    |                        |          |   |          |
|--------------------|------------------------|----------|---|----------|
| Bretagne           | EcoBât Solutions       | 30000.0  | 3 | 51000.0  |
| Hauts-de-France    | Ferme Éolienne du Nord | 350000.0 | 1 | 350000.0 |
| Nouvelle-Aquitaine | Coopérative AgriVert   | 62000.0  | 1 | 62000.0  |
| Occitanie          | SolarTech Industries   | 120000.0 | 1 | 120000.0 |

## Exercice 3 — CASE WHEN - statut lisible

**Notion : CASE WHEN ... THEN ... ELSE ... END**

■ **Objectif** : Transformer les codes de statut bruts en libellés compréhensibles pour un rapport destiné à des non-techniciens.

```
SELECT nom,
       CASE statut
         WHEN 'accepté' THEN 'Financement validé'
         WHEN 'refusé' THEN 'Financement rejeté'
         WHEN 'en instruction' THEN 'En cours d\'analyse'
         ELSE 'Dossier récent, non traité'
       END AS statut_lisible
FROM dossiers_aide d JOIN organismes o ON d.organisme_id = o.id;
```

**Explication** : CASE statut WHEN valeur THEN ... est une variante compacte de CASE WHEN colonne = valeur THEN ..., utile quand on compare toujours la même colonne.

| organisme              | statut_lisible             |
|------------------------|----------------------------|
| EcoBât Solutions       | Financement validé         |
| SolarTech Industries   | En cours d'analyse         |
| Ville de Rennes        | Financement validé         |
| Ferme Éolienne du Nord | Dossier récent, non traité |
| Coopérative AgriVert   | Financement rejeté         |
| EcoBât Solutions       | En cours d'analyse         |

## Exercice 4 — COALESCE - montant et décision par défaut

**Notion : COALESCE()**

■ **Objectif** : Éviter d'afficher des valeurs vides (NULL) pour les dossiers pas encore décidés, en les remplaçant par une valeur ou un texte par défaut.

```
SELECT o.nom, d.montant_demande,
       COALESCE(d.montant_accorde, 0) AS montant_accorde_affiche,
       COALESCE(d.date_decision::TEXT, 'Décision en attente') AS decision
FROM dossiers_aide d JOIN organismes o ON d.organisme_id = o.id;
```

**Explication** : COALESCE retourne la première valeur non NULL. Utile pour éviter d'afficher des champs vides dans un rapport.

| organisme            | montant_demande | montant_accorde_affiche | decision            |
|----------------------|-----------------|-------------------------|---------------------|
| EcoBât Solutions     | 45000.0         | 38000.0                 | 2026-03-01          |
| SolarTech Industries | 120000.0        | 0                       | Décision en attente |

|                        |          |         |                     |
|------------------------|----------|---------|---------------------|
| Ville de Rennes        | 78000.0  | 78000.0 | 2026-02-10          |
| Ferme Éolienne du Nord | 350000.0 | 0       | Décision en attente |
| Coopérative AgriVert   | 62000.0  | 0.0     | 2026-03-20          |
| EcoBât Solutions       | 30000.0  | 0       | Décision en attente |

## Exercice 5 — NOT EXISTS - organismes sans dossier accepté

**Notion :** NOT EXISTS (sous-requête corrélée)

■ **Objectif :** Identifier les organismes qui n'ont encore reçu aucune validation, pour un suivi ciblé de leur dossier.

```
SELECT o.nom
FROM organismes o
WHERE NOT EXISTS (
    SELECT 1 FROM dossiers_aide d
    WHERE d.organisme_id = o.id AND d.statut = 'accepté'
);
```

**Explication :** Pour chaque organisme, on vérifie qu'aucun dossier accepté ne lui est lié. Plus sûr que NOT IN en cas de valeurs NULL.

Résultat attendu : SolarTech Industries, Ferme Éolienne du Nord, Coopérative AgriVert

## Exercice 6 — JSONB - extraire la puissance des projets solaires

**Notion :** ->, ?, cast ::NUMERIC

■ **Objectif :** Extraire une donnée technique précise (puissance en kWc) stockée dans un champ JSON à structure variable, sans avoir besoin d'une colonne dédiée dans la table.

```
SELECT o.nom, d.details ->> 'technologie' AS technologie,
       (d.details ->> 'puissance_kwc')::NUMERIC AS puissance_kwc
FROM dossiers_aide d JOIN organismes o ON d.organisme_id = o.id
WHERE d.details ? 'puissance_kwc'
ORDER BY puissance_kwc DESC;
```

**Explication :** ->> extrait une valeur JSON en texte, converti ensuite en NUMERIC pour permettre le tri numérique. L'opérateur ? vérifie la présence de la clé.

| organisme            | puissance_kwc |
|----------------------|---------------|
| SolarTech Industries | 250           |
| EcoBât Solutions     | 80            |

## Exercice 7 — ARRAY - dossiers tagués 'production'

**Notion :** ANY(tableau)

■ **Objectif :** Retrouver rapidement tous les dossiers liés à une thématique donnée grâce à leurs tags, sans jointure supplémentaire.

```
SELECT o.nom, d.tags
FROM dossiers_aide d JOIN organismes o ON d.organisme_id = o.id
WHERE 'production' = ANY(d.tags);
```

**Explication :** ANY() vérifie si la valeur recherchée figure dans le tableau, quelle que soit sa position.

Résultat attendu : SolarTech Industries, Ferme Éolienne du Nord

## Exercice 8 — Index - accélérer la recherche par SIRET

**Notion :** CREATE INDEX, EXPLAIN

■ **Objectif :** Vérifier que les recherches fréquentes (par SIRET, par organisme, par statut) s'appuient bien sur un index plutôt que sur un parcours complet de la table.

```
CREATE INDEX idx_dossiers_organisme ON dossiers_aide (organisme_id);
CREATE INDEX idx_dossiers_agent ON dossiers_aide (agent_id);
CREATE INDEX idx_dossiers_statut ON dossiers_aide (statut);
```

```
EXPLAIN SELECT * FROM organismes WHERE siret = '23456789000122';
```

**Explication :** siret est déjà indexé automatiquement (UNIQUE). Les clés étrangères, elles, ne le sont pas par défaut : on les indexe manuellement si elles sont souvent utilisées en jointure ou filtre.

| resultat                                                 |
|----------------------------------------------------------|
| Index Scan using organismes_siret_key on organismes      |
| Index Cond: (siret = '23456789000122'::text)             |
| cost=0.15..8.17 rows=1 (estimation, valeurs indicatives) |

## Exercice 9 — Transaction - valider un dossier en instruction

**Notion :** BEGIN / COMMIT

■ **Objectif :** Faire passer un dossier de 'en instruction' à 'accepté' en garantissant que le changement de statut ET le montant accordé soient appliqués ensemble, ou pas du tout en cas d'erreur.

```
BEGIN;

UPDATE dossiers_aide
SET statut = 'accepté', montant_accorde = 100000.00, date_decision = CURRENT_DATE
WHERE organisme_id = (SELECT id FROM organismes WHERE nom = 'SolarTech Industries')
AND statut = 'en instruction';

SELECT * FROM dossiers_aide
WHERE organisme_id = (SELECT id FROM organismes WHERE nom = 'SolarTech Industries');

COMMIT;
```

**Explication :** L'UPDATE et sa vérification se font à l'intérieur du même bloc transactionnel ; COMMIT valide définitivement, ROLLBACK aurait tout annulé.

| organisme            | montant_demande | montant_accorde | statut  | date_decision |
|----------------------|-----------------|-----------------|---------|---------------|
| SolarTech Industries | 120000.0        | 100000.0        | accepté | date du jour  |

## Exercice 10 — Vue - tableau de bord des dossiers par catégorie

Notion : CREATE VIEW

■ **Objectif** : Créer un rapport réutilisable donnant en un coup d'œil le nombre de dossiers et les montants totaux par catégorie d'aide, sans avoir à réécrire la requête à chaque consultation.

```
CREATE VIEW bilan_categories AS
SELECT c.nom AS categorie,
       COUNT(d.id) AS nombre_dossiers,
       COALESCE(SUM(d.montant_demande), 0) AS total_demande,
       COALESCE(SUM(d.montant_accorde), 0) AS total_accorde
FROM categories_aides c
LEFT JOIN dossiers_aide d ON d.categorie_id = c.id
GROUP BY c.nom;

SELECT * FROM bilan_categories ORDER BY total_demande DESC;
```

**Explication** : LEFT JOIN conserve les catégories sans aucun dossier, avec des totaux à 0 grâce à COALESCE. La vue est réutilisable dans n'importe quelle requête future.

| categorie              | nombre_dossiers | total_demande | total_accorde |
|------------------------|-----------------|---------------|---------------|
| Éolien                 | 1               | 350000.0      | 0.0           |
| Solaire photovoltaïque | 2               | 150000.0      | 100000.0      |
| Efficacité énergétique | 1               | 78000.0       | 78000.0       |
| Énergies renouvelables | 1               | 62000.0       | 0.0           |
| Isolation              | 1               | 45000.0       | 38000.0       |
| Bâtiment               | 0               | 0.0           | 0.0           |
| Énergie                | 0               | 0.0           | 0.0           |

## Exercice 11 — PARTIE 2 - Taux d'engagement par catégorie et par année

Notion : COALESCE, calcul dérivé, LEFT JOIN

■ **Objectif** : Mesurer, pour chaque catégorie d'aide, quelle proportion du budget alloué a déjà été engagée par des décisions favorables.

```
SELECT c.nom AS categorie, e.annee, e.budget_alloue,
       COALESCE(SUM(d.montant_accorde), 0) AS montant_engage,
       ROUND(COALESCE(SUM(d.montant_accorde), 0) / e.budget_alloue * 100, 2) AS taux_engagement_pct
FROM enveloppes_budgetaires e
JOIN categories_aides c ON e.categorie_id = c.id
LEFT JOIN dossiers_aide d ON d.categorie_id = e.categorie_id AND d.statut = 'accepté'
GROUP BY c.nom, e.annee, e.budget_alloue
ORDER BY taux_engagement_pct DESC;
```

**Explication** : Le taux d'engagement rapporte le montant déjà accordé au budget alloué de l'enveloppe. COALESCE évite une division par NULL pour les catégories sans dossier accepté.

| categorie              | budget_alloue | engage | taux_pct |
|------------------------|---------------|--------|----------|
| Efficacité énergétique | 200000        | 78000  | 39.0     |
| Isolation              | 150000        | 38000  | 25.33    |



|                        |        |        |      |
|------------------------|--------|--------|------|
| Solaire photovoltaïque | 400000 | 100000 | 25.0 |
| Énergies renouvelables | 500000 | 0      | 0.0  |
| Éolien                 | 600000 | 0      | 0.0  |

## Exercice 12 — PARTIE 2 - Reste à verser par dossier accepté

Notion : LEFT JOIN, SUM, COALESCE

■ **Objectif** : Suivre, pour chaque dossier validé, le montant qu'il reste concrètement à payer à l'organisme bénéficiaire.

```
SELECT o.nom AS organisme, d.montant_accorde,
       COALESCE(SUM(v.montant), 0) AS deja_verse,
       d.montant_accorde - COALESCE(SUM(v.montant), 0) AS reste_a_verser
FROM dossiers_aide d
JOIN organismes o ON d.organisme_id = o.id
LEFT JOIN versements v ON v.dossier_id = d.id
WHERE d.statut = 'accepté'
GROUP BY o.nom, d.montant_accorde
ORDER BY reste_a_verser DESC;
```

**Explication** : Le reste à verser est la différence entre le montant accordé et la somme des versements déjà effectués pour ce dossier.

| organisme            | montant_accorde | deja_verse | reste_a_verser |
|----------------------|-----------------|------------|----------------|
| SolarTech Industries | 100000.0        | 50000.0    | 50000.0        |
| Ville de Rennes      | 78000.0         | 78000.0    | 0.0            |
| EcoBât Solutions     | 38000.0         | 38000.0    | 0.0            |

## Exercice 13 — PARTIE 2 - Cumul progressif des versements

Notion : SUM() OVER (ORDER BY ...)

■ **Objectif** : Visualiser l'évolution de la trésorerie réellement décaissée au fil du temps, tous dossiers confondus.

```
SELECT v.date_versement, o.nom AS organisme, v.montant,
       SUM(v.montant) OVER (ORDER BY v.date_versement) AS montant_cumule_verse
FROM versements v
JOIN dossiers_aide d ON v.dossier_id = d.id
JOIN organismes o ON d.organisme_id = o.id
ORDER BY v.date_versement;
```

**Explication** : Fonction de fenêtrage sans PARTITION BY : le cumul porte sur l'ensemble des versements, triés chronologiquement - utile pour suivre la trésorerie décaissée dans le temps.

| date       | organisme            | montant | cumule   |
|------------|----------------------|---------|----------|
| 2026-02-20 | Ville de Rennes      | 78000.0 | 78000.0  |
| 2026-03-15 | EcoBât Solutions     | 19000.0 | 97000.0  |
| 2026-06-10 | SolarTech Industries | 50000.0 | 147000.0 |
| 2026-06-15 | EcoBât Solutions     | 19000.0 | 166000.0 |

## Exercice 14 — PARTIE 2 - Prévisionnel et niveau d'alerte budgétaire

Notion : CASE WHEN sur un ratio calculé

■ **Objectif** : Anticiper le risque de dépassement budgétaire en classant chaque catégorie selon son niveau d'engagement actuel.

```
SELECT c.nom AS categorie, e.budget_alloue,
       COALESCE(SUM(d.montant_accorde), 0) AS engage,
       e.budget_alloue - COALESCE(SUM(d.montant_accorde), 0) AS budget_restant,
       CASE
         WHEN COALESCE(SUM(d.montant_accorde), 0) / e.budget_alloue >= 0.8 THEN 'Alerte - budg'
         WHEN COALESCE(SUM(d.montant_accorde), 0) / e.budget_alloue >= 0.4 THEN 'Vigilance - e'
         ELSE 'Marge disponible confortable'
       END AS niveau_alerte
FROM enveloppes_budgetaires e
JOIN categories_aides c ON e.categorie_id = c.id
LEFT JOIN dossiers_aide d ON d.categorie_id = e.categorie_id AND d.statut = 'accepté'
GROUP BY c.nom, e.budget_alloue
ORDER BY budget_restant ASC;
```

**Explication** : CASE WHEN classe chaque catégorie selon son taux d'engagement, pour un pilotage visuel simple sans outil externe.

| categorie              | budget_alloue | engage   | budget_restant | niveau_alerte                |
|------------------------|---------------|----------|----------------|------------------------------|
| Éolien                 | 600000.0      | 0.0      | 600000.0       | Marge disponible confortable |
| Énergies renouvelables | 500000.0      | 0.0      | 500000.0       | Marge disponible confortable |
| Solaire photovoltaïque | 400000.0      | 100000.0 | 300000.0       | Marge disponible confortable |
| Efficacité énergétique | 200000.0      | 78000.0  | 122000.0       | Marge disponible confortable |
| Isolation              | 150000.0      | 38000.0  | 112000.0       | Marge disponible confortable |

## Exercice 15 — PARTIE 2 - Vue tableau de bord financier global

Notion : CREATE VIEW, sous-requête corrélée dans SELECT

■ **Objectif** : Regrouper en une seule vue réutilisable les indicateurs clés (budget, engagé, versé, taux) pour un reporting financier régulier.

```
CREATE VIEW tableau_bord_financier AS
SELECT c.nom AS categorie, e.annee, e.budget_alloue,
       COALESCE(SUM(DISTINCT d.montant_accorde), 0) AS montant_engage,
       COALESCE((SELECT SUM(v.montant) FROM versements v
                  JOIN dossiers_aide d2 ON v.dossier_id = d2.id
                  WHERE d2.categorie_id = e.categorie_id), 0) AS montant_verse,
       ROUND(COALESCE(SUM(DISTINCT d.montant_accorde), 0) / e.budget_alloue * 100, 2) AS taux_eng
FROM enveloppes_budgetaires e
JOIN categories_aides c ON e.categorie_id = c.id
LEFT JOIN dossiers_aide d ON d.categorie_id = e.categorie_id AND d.statut = 'accepté'
GROUP BY c.nom, e.annee, e.budget_alloue, e.categorie_id;

SELECT * FROM tableau_bord_financier ORDER BY taux_engagement_pct DESC;
```

**Explication** : Combine budget, montant engagé (via LEFT JOIN) et montant versé (via sous-requête corrélée) en une seule vue de synthèse, réutilisable pour un reporting régulier.

| categorie              | annee | budget_alloue | montant_engage | montant_verse | taux_pct |
|------------------------|-------|---------------|----------------|---------------|----------|
| Efficacité énergétique | 2026  | 200000.0      | 78000.0        | 78000.0       | 39.0     |
| Isolation              | 2026  | 150000.0      | 38000.0        | 38000.0       | 25.33    |
| Solaire photovoltaïque | 2026  | 400000.0      | 100000.0       | 50000.0       | 25.0     |
| Énergies renouvelables | 2026  | 500000.0      | 0.0            | 0.0           | 0.0      |
| Éolien                 | 2026  | 600000.0      | 0.0            | 0.0           | 0.0      |

---

### 3. Points importants à retenir

- L'ordre d'écriture d'une requête reste toujours SELECT > FROM > JOIN > WHERE > GROUP BY > HAVING > ORDER BY > LIMIT.
- PostgreSQL diffère d'autres SGBD sur SERIAL, ILIKE, LIMIT/OFFSET, JSONB/ARRAY, RETURNING et les identifiants sensibles à la casse.
- WHERE s'applique toujours avant ORDER BY, y compris après une CTE.

## 4. Perspectives — KPI de pilotage budgétaire

Ce projet peut être enrichi pour établir tous les KPI de pilotage budgétaire pertinents, permettant de prendre des décisions optimales, notamment :

1. Taux d'engagement budgétaire par catégorie/région/année
2. Délai moyen d'instruction (dépôt -> décision)
3. Taux d'acceptation des dossiers (accepté / total traité)
4. Reste à verser global et par échéance (prévision de trésorerie)
5. Montant moyen accordé par dossier, par secteur d'activité

**Choix d'outil :** Pour l'exploitation et la visualisation de ces indicateurs, nous privilégions Power BI plutôt que des requêtes SQL de reporting : tableaux de bord interactifs, filtres dynamiques et partage facilité auprès de décideurs non techniques.